

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 07

NOMBRE COMPLETO: Cruz Cano Omar

Nº de Cuenta: 318068932

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027 - 1

FECHA DE ENTREGA LÍMITE: 14 de octubre de 2026

CALIFICACIÓN: _____

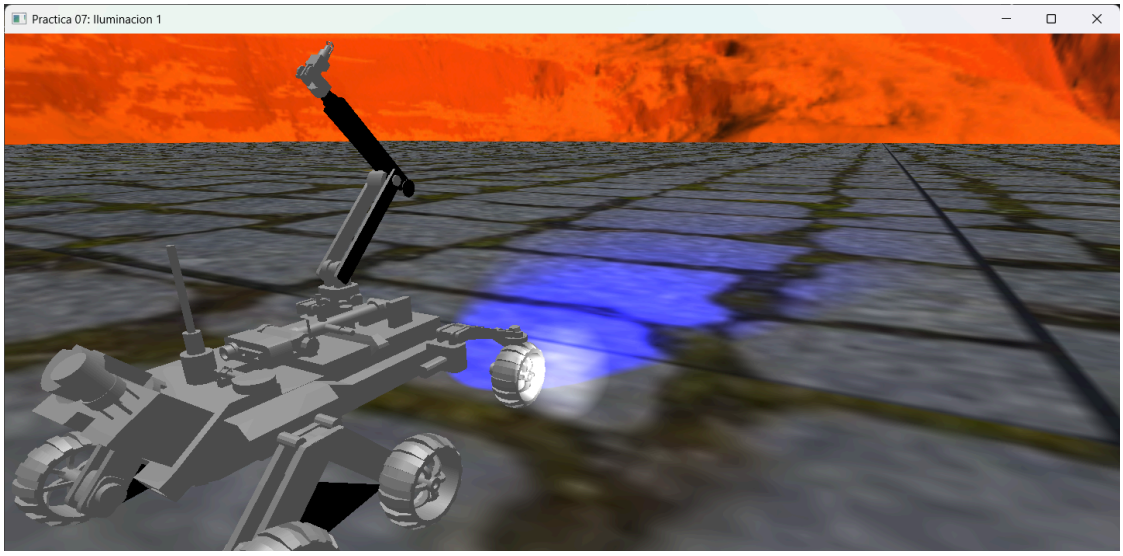
Añadir una luz spotlight frontal al rover que ilumine hacia adelante y al avanzar o retroceder, la luz siga unida a él.

Para añadir esta luz fue necesario configurarla desde la declaración de la misma, primero le asignamos un color azul; luego ajustamos la **aIntensity** en 1.0 y la **dIntensity** en 10.0 para lograr que ilumine fuertemente al inicio de la luz; la posición la dejé en 0,0,0 para la declaración; para lograr que la luz apunte hacia adelante y hacia abajo del rover asigné en **xdir** el valor de 3.0 y en **ydir** un valor de -1.0; para la atenuación dejamos el valor de **con** en 1.0 para garantizar que la luz tenga el 100% de su intensidad en su punto de origen, para el valor de **lin** usamos 0.0 para eliminar por completo el desgaste proporcional a cortas distancias ya que se utilizó un valor de **exp** de 0.2 el cual a cierta distancia destruye la intensidad rápidamente hacia el cero, esto garantiza una iluminación fuerte al inicio y que se oscurece de golpe a la distancia exacta en lugar de desvanecerse lentamente por todo el mapa; finalmente, dejamos el valor de **edg** en 30° para lograr el efecto de que ilumina el camino por delante como los faros de un auto.

```
// luz frente al rover
spotLights[3] = SpotLight(0.0f, 0.0f, 1.0f,
    1.0f, 10.0f,
    0.0f, 0.0f, 0.0f,
    3.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.02f,
    30.0f);
spotLightCount++;
```

```
// Rover
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getMueveRoverX(), -1.0f, -7.0));
spotLights[3].SetPos(glm::vec3(0.9f + mainWindow.getMueveRoverX(), 0.3f, -7.0));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Rover_M.RenderModel();
```

Para lograr que el rover se moviera sobre el eje X, fue necesario sumarle en la translación en el eje correspondiente un valor llamado **mueveRoverX** que se manipula desde el teclado utilizando las teclas **B** y **N**; Después haciendo uso de la función .**setPos** de nuestro spotLight, le dimos una posición inicial acorde al modelo para simular que la luz sale de la parte delantera del rover y también se le suma en el eje correspondiente el valor que se manipula con el teclado para que la luz se quede pegada al rover sin importar a dónde se mueva.



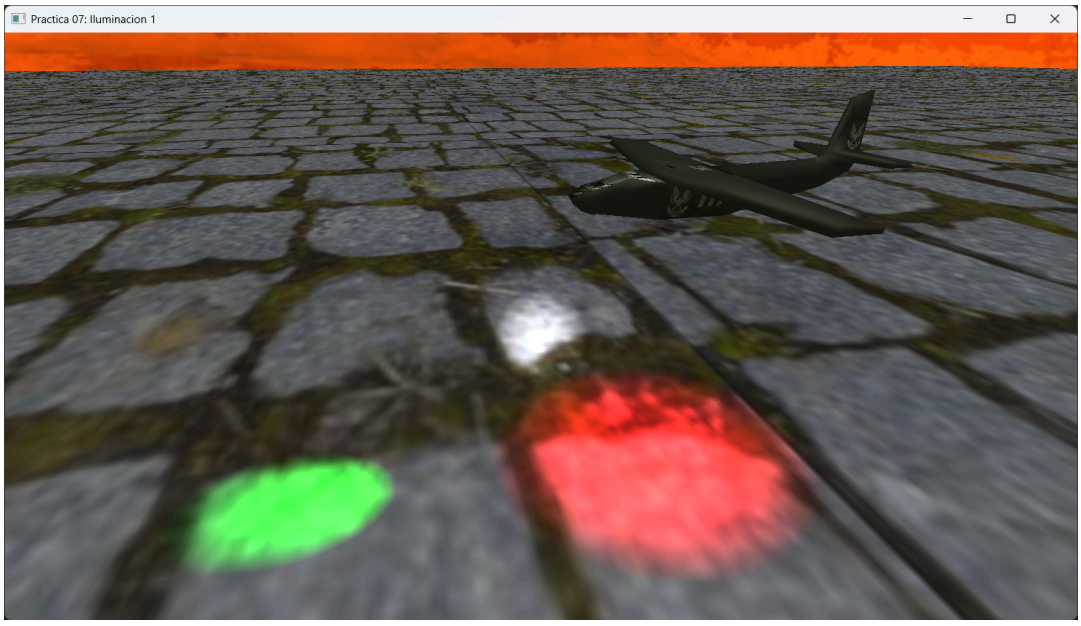
Añadir una luz spotlight al avión que ilumine hacia abajo y al subir y bajar, la luz siga unida a él.

Añadir una luz spotlight al avión que ilumine hacia adelante y al avanzar o retroceder, la luz siga unida a él.

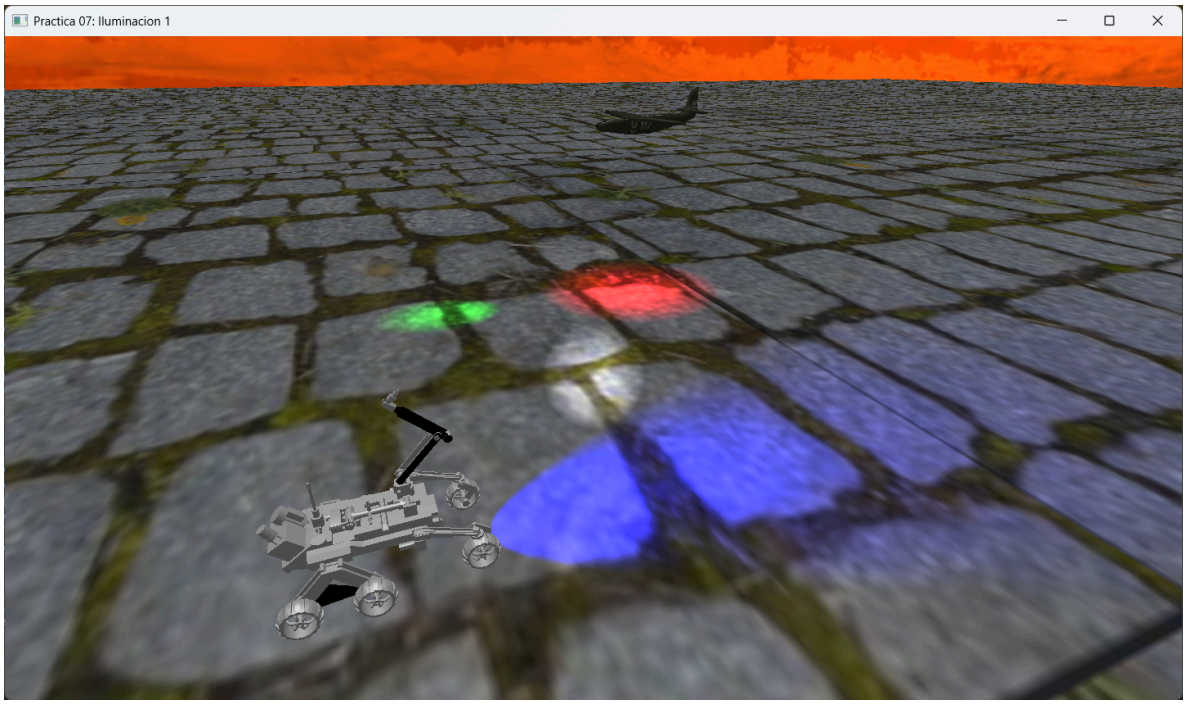
En el caso del avión, utilicé un modelo texturizado por mí en blender y repitiendo la lógica del rover le agregué movimiento en el eje **X** y en el eje **Y** usando las variables **muevePlaneX** y **muevePlaneY** que se manipulan con el teclado; con las teclas **T** y **G** podemos mover el avión hacia arriba y hacia abajo, mientras que las teclas **H** y **J** lo mueven hacia adelante y hacia atrás. Para la luz roja que apunta hacia abajo utilizamos un **xdir** de -1.0 mientras que los otros dos valores se mantienen en 0; la atenuación tiene un valor de 1.0 en la constante, un valor de 0.0 en en el factor lineal y un valor de 0.04 en el factor exponencial. Para la luz verde que apunta hacia adelante y abajo utilizamos un **xdir** y un **ydir** de -1 y para los valores de atenuación cambiamos el factor exponencial a un valor de 0.05 y un valor de **edg** de solamente 10 grados para simular una luz con un cono más cerrado para apuntar a objetivos en específico.

```
// Avion
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getMuevePlaneX(), 6.0f + mainWindow.getMuevePlaneY(), -20.0));
spotLights[1].SetPos(glm::vec3(-0.5f + mainWindow.getMuevePlaneX(), 6.0f + mainWindow.getMuevePlaneY(), -20.0));
spotLights[2].SetPos(glm::vec3(-1.5f + mainWindow.getMuevePlaneX(), 6.0f + mainWindow.getMuevePlaneY(), -20.0));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Plane_M.RenderModel();
```

Para lograr que la luz se quede pegada al avión utilizamos las mismas variables **muevePlaneX** y **muevePlaneY** para sumarlas a la posición de la luz en el eje correcto; la luz verde se encuentra en la punta del avión mientras que la luz roja se encuentra casi en el centro del modelo.



Por último, se hicieron las modificaciones pertinentes al **Window.cpp** para cambiar el nombre de la ventana y se mantuvo la luz tipo spotlight que corresponde a la linterna y la luz direccional de color blanco que simula la posición del sol a medio día. También se hizo una modificación al evento de la letra **G** para que el modelo del avión no atraviese el suelo al llegar a un valor específico.



Conclusión

La elaboración de esta práctica fue fundamental para comprender a profundidad la configuración matemática y el comportamiento de las diversas fuentes de luz en un entorno gráfico, desde la iluminación global hasta los haces enfocados; A través de la manipulación detallada de parámetros como las intensidades, la dirección vectorial y el ángulo de apertura, fue posible simular efectos realistas, tales como el faro frontal de un vehículo o los reflectores de un avión, complementando con la luz direccional blanca que asume el rol del sol.

Un aprendizaje técnico significativo radicó en el control de la atenuación de la luz; al ajustar los factores constante, lineal y exponencial, logramos dominar cómo un spotlight puede mantenerse intenso en su punto de origen y decaer de forma abrupta a la distancia, permitiendo una personalización precisa del impacto visual sobre los modelos texturizados.

Adicionalmente, logramos integrar la interacción humano-computadora al vincular las variables de traslación del teclado con la posición de los spotlights y los modelos, garantizando que la iluminación se mantuviera anclada físicamente al rover y al avión durante su desplazamiento, esta correcta conexión demostró ser esencial para consolidar el realismo e interactividad de la escena.